



Introducing Meerkat: an experiment in global consensus

2026-07-08



James Larisch



Bob Halley



João Pedro Leite

12 min read



Many internal services at Cloudflare need to read and modify the same control-plane state from across our 330+ global data centers. They need guarantees that different readers *never* see inconsistent state, and that the system remains available for writes even when some data centers or links fail.

But Cloudflare's network runs across the entire Internet, and the Internet is an unpredictable place. Servers and data centers go down. Queues fill up. Links and cables get cut. These conditions make it difficult to run a globally available data system that guarantees strong consistency (e.g., that all readers are guaranteed to read all prior writes) because hostile conditions hinder distributed system replicas' ability to reliably synchronize data with one another.

One way to synchronize data safely despite adverse network conditions is via a *consensus algorithm*, which allows a set of machines to agree on the same sequence of values, such as key-value store put and get operations, as long as a majority remains alive and able to communicate.

Unfortunately, commonly deployed consensus algorithms like [Raft](#) suffer in wide-area networks like Cloudflare's because they rely on *leaders* and *timeouts*. The *leader* is the only replica allowed to make writes, and if it fails due to a crash or network degradation, the system becomes unavailable until some other replica *times out* and a new leader is elected. And these timeout values are hard to configure in networks with unpredictable latencies.

We have experienced multiple incidents caused by unavailable leaders in consensus-driven systems.

And so, for the past year, Cloudflare's Research [team](#) has been building a new distributed consensus service called **Meerkat** powered by a consensus algorithm called [QuePaxa](#), published in 2023 by Tennage & Băscescu et al. QuePaxa differs from Raft in that all replicas can perform writes at all times, and progress is never halted due to a timeout, which makes it well suited for Cloudflare's network. We layer *applications*, like a transactional key-value store and leasing system, atop Meerkat's consensus log. To our knowledge, this will be the first industrial deployment of QuePaxa at global scale.

Meerkat is an experimental consensus service that is still in development. It's being designed initially to manage small pieces of control plane state (e.g., leadership for replicated databases) and so it will be kept internal-only for the immediate future. This post introduces Meerkat and lays the groundwork for the Meerkat-related blog posts to come.

What we need from a global control-plane data system [↗](#)

Many Cloudflare services read and write *control-plane data*, data that helps those services operate correctly, from multiple machines distributed all over the world. One example of control-plane data is *placement information*: where certain resources (like an AI model instance) are stored. Another example is *leadership information*: which machine is currently allowed to perform writes to a database.

Control-plane data must be both *strongly consistent* and *accessible despite particular kinds of faults*.

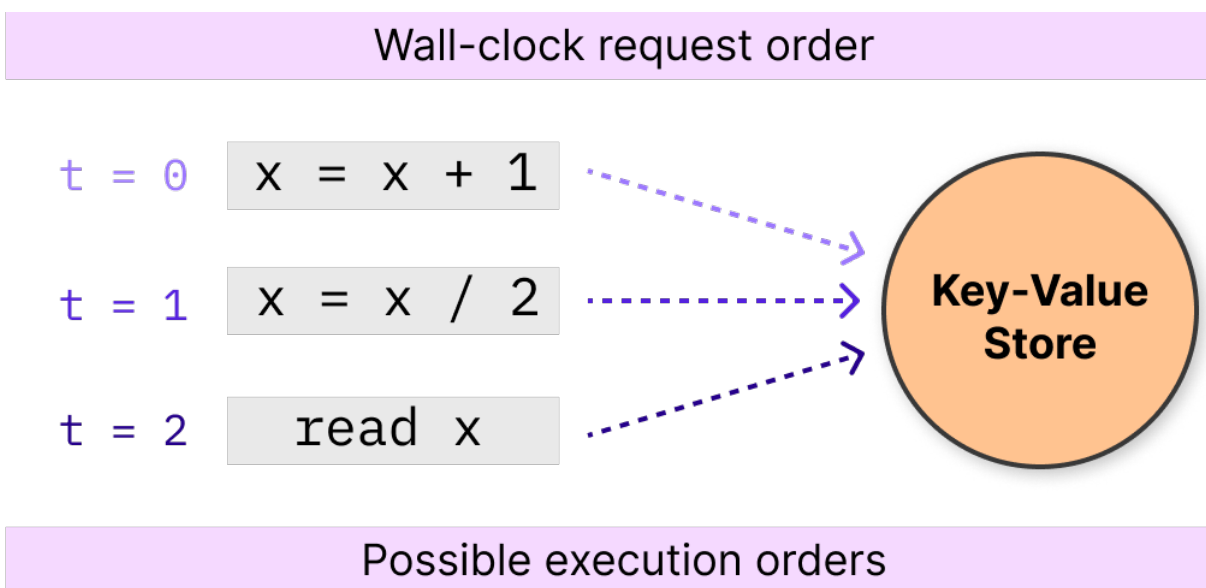
In this section we precisely describe our consistency and fault tolerance requirements for a Cloudflare consensus service. We use a key-value store for a running example of an application running atop our consensus service, though other applications (e.g., distributed leases/locks) are possible.

Strong consistency [↗](#)

A distributed data system's [consistency](#) level describes what kinds of weird behavior the system is allowed to exhibit when it receives concurrent reads and writes. Consider a distributed key-value store that stores a single numeric value $x = 6$ across multiple nodes. Also consider the following sequence of writes. These writes are submitted to different nodes on a best-effort basis, and could arrive in any order:

1. $x = x + 1$
2. $x = x / 2$

A system's consistency level tells you what values of x a client might see when reading x after these writes. Consider the following sequence of operations and the possible execution orders under different consistency levels:



Weak consistency:
arbitrary order

operation	x
<i>(start)</i>	6
$x = x / 2$	3
read x	(3)
$x = x + 1$	4

Stronger consistency:
writes ordered

operation	x
<i>(start)</i>	6
$x = x + 1$	7
read x	(7)
$x = x / 2$	3.5

Strongest consistency:
ordered by t

operation	x
<i>(start)</i>	6
$x = x + 1$	7
$x = x / 2$	3.5
read x	(3.5)

In a weak consistency level, writes can be re-ordered. In a stronger consistency model, writes can't be reordered, but reads can. In the strongest possible consistency level, the operations are ordered exactly as they

occurred in real time. This property is called *linearizability*.

At Cloudflare, many services want linearizability. Unlike weaker forms of consistency, linearizability relieves programmers from thinking about all the weird behaviors the data systems might exhibit. Instead, they can reason about the distributed system like they reason about local memory on a single-threaded machine: all reads after a write will see that write. For additional reading material on the dangers of weak consistency, check out this [post](#) by Marc Brooker.

(If you're wondering, Meerkat's key-value store also provides serializability, which we'll write about in a future post.)

Fault tolerance [🔗](#)

A system's level of fault tolerance describes what kinds of faults the system can handle before catastrophes happen. Catastrophes are typically violations of properties the system aims to uphold, e.g., that two consecutive reads without an intervening write for the same key never see different values, or that the system remains available for writes. The faults include network failures or delays, machine crashes, and machine restarts. A system will typically explicitly handle some faults but not others (you can't handle all faults, as the universe could always reach heat-death). For example, some key-value stores might guarantee to remain available for writes as long as two-thirds of the machines in the system can communicate and don't crash, but make no promises if a machine is compromised and starts sending malicious messages.

Our desired fault tolerance properties are as follows:

First, the data system should remain available for writes and reads from a client located in any of our data centers as long as the following are true:

1. A majority of the machines in our system are alive and can communicate with one another. (Formally, we tolerate f faults in a system of $2f + 1$ machines).
2. The client can contact *any* machine in the system that is connected to a majority of live machines.

This means that a single failed machine, or network degradation on a single link, does not affect availability of the system. This property is not provided by Raft-based systems, as we'll see later.

Second, the data system remains *correct* as long as no actor in the system is actively malicious (and, of course, there are no bugs). We define *correctness* in terms of consensus *safety* later, but loosely speaking this means no two up-to-date machines will ever disagree about the world (e.g., one thinks that $\text{key1}=1$ while another thinks that $\text{key1}=2$).

To summarize, the system must remain correct even if machines crash, machines restart, networks fail or degrade, data centers go down, and more (though we, like Raft-based systems, do not handle [Byzantine faults](#)).

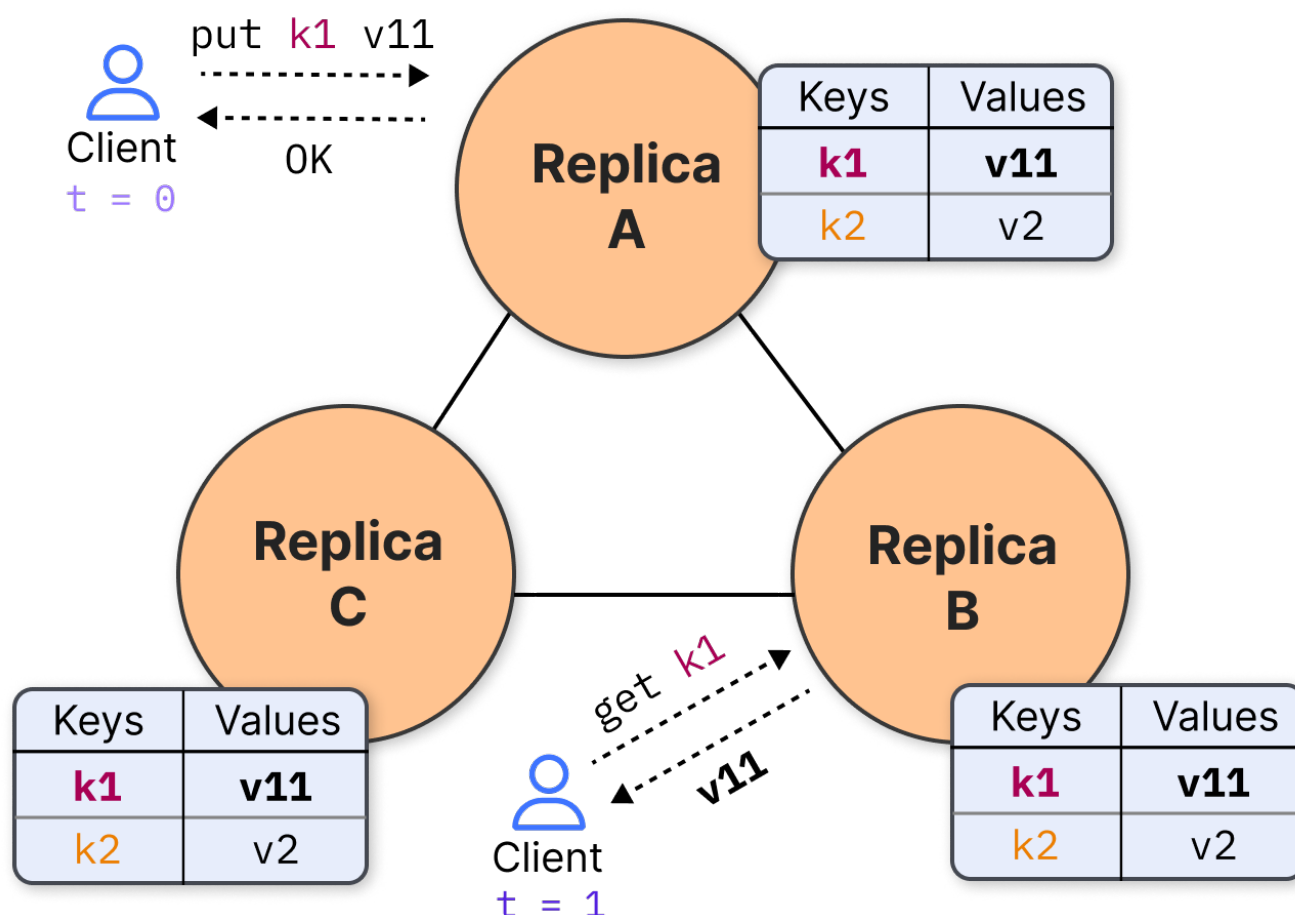
Introducing Meerkat [↗](#)

Meerkat is a consensus service *upon which* we can build applications that exhibit the above properties (strong consistency and fault tolerance) like a key-value (KV) store. To understand how Meerkat works, we first outline Meerkat's general architecture, and then describe how Meerkat's choice of consensus algorithm helps provide strong consistency and fault tolerance.

Developers of services using Meerkat request a *cluster* of Meerkat *replicas*. Each replica is connected to every other replica. Each replica participates in the consensus algorithm and can receive both reads and writes. The

developer can specify which data centers are allowed to host their replicas, and Meerkat places them automatically.

To interact with their cluster, a developer's client sends an application-specific request to any replica in the cluster. A single replica may host many kinds of applications, but the simplest one is a key-value store, so the simplest application-specific request type is a KV `get` or `put`. The replica responds to the request with an application-specific response (e.g., the records requested with the `get`). Note that KV reads (`get`s) are guaranteed to read up-to-date information.



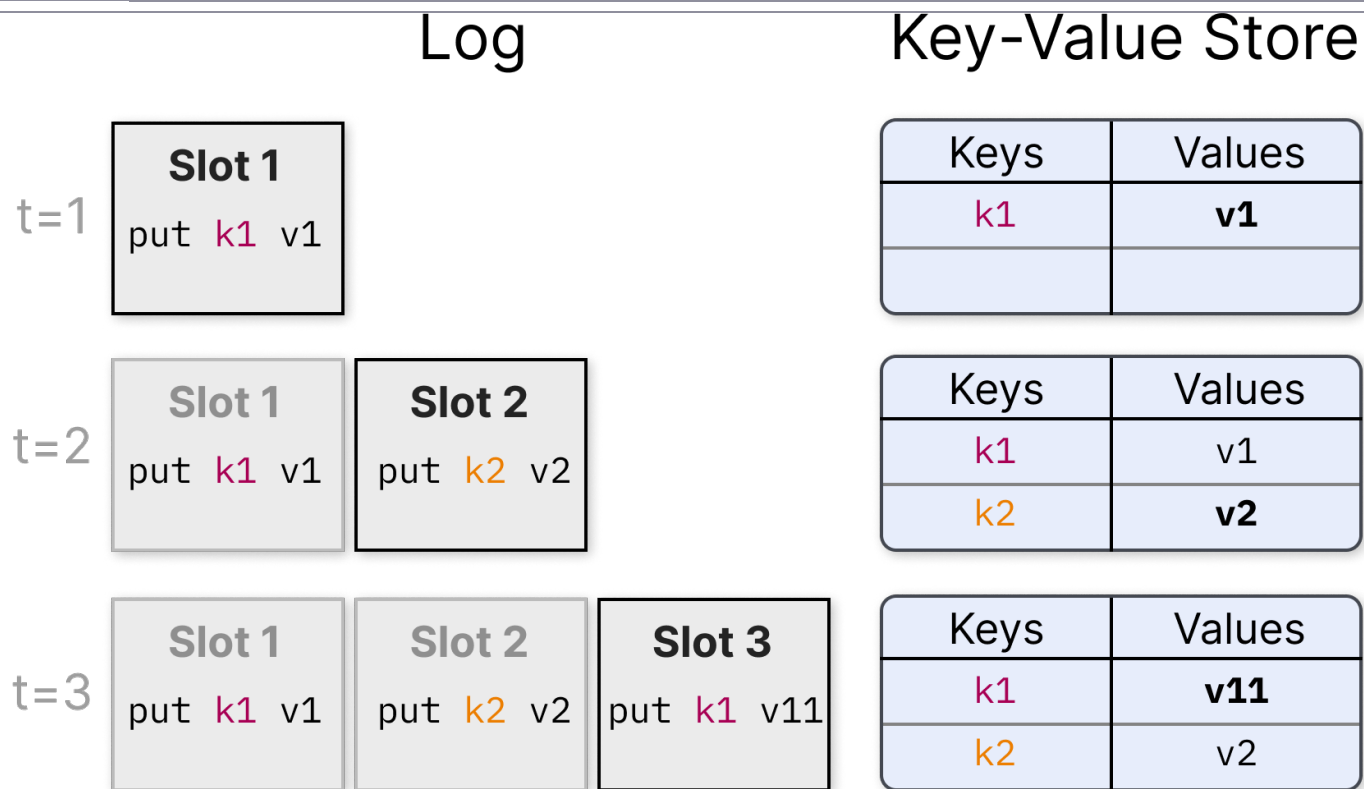
Meerkat's log [↗](#)

Under the hood, the replica translates application requests (e.g., `get` and `put`) into *log events*. That replica distributes each log event to all other

replicas using a consensus algorithm such that all replicas maintain the exact same log of events (in reality, a replica may lag behind, but shall never record different entries). These events are arbitrary — Meerkat’s core doesn’t care what’s in them. Meerkat *applications* care about log event contents. Each Meerkat replica “hosts” many Meerkat applications (e.g., key-value store) that read the log events and construct state. (Note that each replica belongs to exactly one cluster.)

For instance, the KV Meerkat application constructs an in-memory key-value store from the log events. So when a client sends a write like `put k1 v1`, the receiving replica places that write into a log event and distributes it to all replicas. If someone else subsequently writes `put k1 v11` to a different replica, this event is also distributed to all replicas. Since all functioning replicas have the same log, those replicas can apply the operations in the log in sequence to construct the exact same state. Note that `get` requests also create distributed log events (for linearizability, as explained in the next section).

Here is an example of how a replica’s KV store is updated as it receives log events:



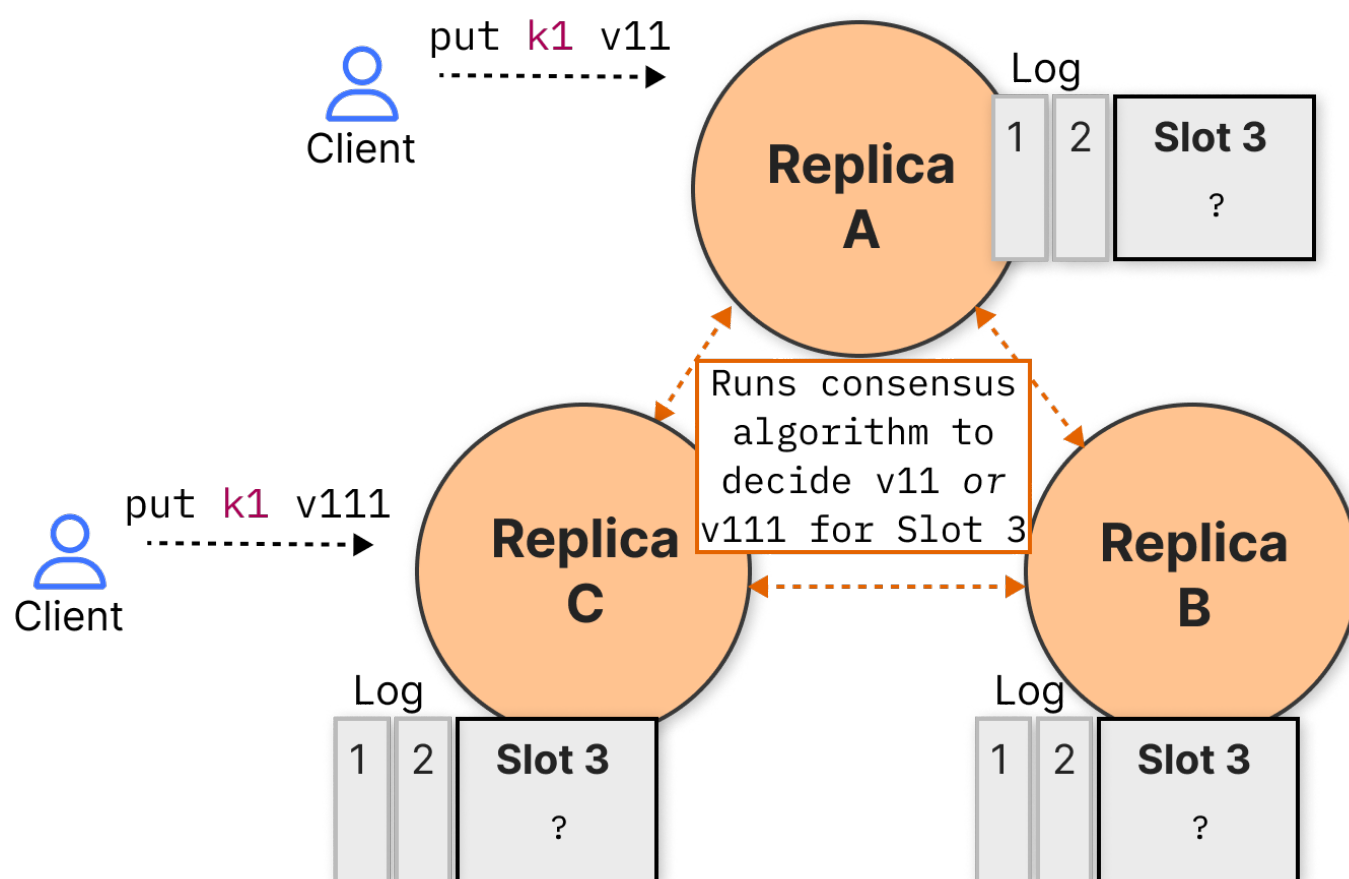
How Meerkat's log enables strong consistency [↗](#)

Meerkat guarantees that if one client executes `put k1 v1`, a second client subsequently executes `put k1 v11`, and a third client subsequently executes `get k1` (with a consistent read), they will always read `v11`. It guarantees this even if each request is submitted to a different replica, and those replicas are distributed randomly across the world. This is linearizability. To see how Meerkat guarantees this, we must examine Meerkat's log in more detail.

The Meerkat log is a sequence of slots. A slot is a box that can contain an event or not. A slot that contains an event is called a *decided* slot. All slots in the log are decided except the last slot, which is currently being decided. One of Meerkat's invariants is that if any two replicas decide on the value for a slot, those values are the same. In other words, no two replicas will ever disagree on the value of a decided slot (though one replica may think the last slot is empty while another does not). This property helps guarantee the desired properties we described in the previous section.

To decide on the value of the last (empty) slot in the log, Meerkat replicas run a distributed *consensus algorithm*. A consensus algorithm allows a set of machines communicating over a network to agree on a decided slot value. Our consensus algorithm works as long as a majority of replicas (more than half) are alive.

So if the log currently contains two entries, and a client submits `put k1 v11` to a replica, that replica triggers a consensus algorithm for slot 3. But another client might have submitted `put k1 v111` to a different replica for slot 3. The consensus algorithm ensures that only one such *proposal* for slot 3 wins out. Specifically, it ensures that at least a majority of replicas agree on the same proposal, *deciding* it for slot 3. The non-majority can *never* decide a different proposal, but might miss the fact that slot 3 has been decided at all.



To see how this provides linearizability for our key-value store, consider a write followed by a read. One replica Z proposes `put k1 v11` and this

proposal is decided at slot 3 by a majority of replicas, but NOT replica Y. Subsequently, a reader executes `get k1` on replica Y. Replica Y believes slot 3 is empty, so proposes `get k1` at slot 3. Critically, a majority of replicas will not agree to place that event at slot 3, because that slot has already been decided. They will force replica Y to decide (by receiving older decisions) `put k1 v11` in slot 3, and to propose `get k1` for slot 4, thus *linearizing* the read after the write in the log. (And if that replica can't contact a majority, it will be unable to complete the read.)

How Meerkat's consensus algorithm provides higher availability than Raft [↗](#)

Deciding on log entries requires a distributed consensus algorithm. But which one? All valid consensus algorithms would provide the required consistency and correctness guarantees, but not all provide the same availability guarantees.

Specifically, many algorithms that rely on authoritative leaders do not provide our desired availability guarantees, because they can become unavailable when a single machine experiences issues. Consider [Raft](#), one of the most well-known and probably *the* most implemented consensus algorithm. Raft relies on an authoritative leader: the only replica in the cluster that can drive consensus. As a result, all writes get forwarded to the leader. This design choice helps make Raft "[understandable](#)" and, coupled with [leases](#), can make leader-served reads automatically linearizable (since they're guaranteed to be up-to-date). But it also adds a single point of (temporary) failure.

In general, there are two problems with authoritative leaders. First, if the leader goes down, the system becomes unavailable (all writes block) until a new leader is elected. This is unacceptable for Meerkat. Second, if the leader stays up but slows down, either because it is overloaded or there are network

delays, then performance degrades. The leader is a bottleneck because there is no alternative way to perform writes.

The first problem is exacerbated in wide-area networks. Consider that when a leader goes down, most algorithms choose a new leader using *timeouts*: if a non-leader replica hasn't heard from the leader in some amount of time, they propose themselves as the leader. At that point, the old leader has been deposed, and the system cannot accept writes until a new leader has been elected. The problem is that when the timeout is shorter than the network delay between the original leader and that replica, replicas will constantly be timing out and thus blocking writes. And when the timeout is too long, the system reacts slowly to a failed leader, during which writes are also blocked. Plus, if multiple replicas propose themselves as leader at the same time, their "campaigns" can interfere with each other, causing them to constantly re-propose themselves as leader — all the while blocking writes. We have seen these exact issues with Cloudflare's systems that use Raft because our wide-area network delays can and do vary wildly, making tuning timeouts especially difficult.

We chose a different consensus algorithm for Meerkat, called [QuePaxa](#), that aims to avoid the "tyranny of timeouts" imposed by protocols like Raft. QuePaxa is a subtle protocol, but here are the highlights. A client can contact *any* replica, and that replica can drive consensus for the latest slot. There is a leader, but it is not required — its only advantage is that it can drive consensus with fewer round trips (one) than other replicas (3+). Critically, clients are free to contact multiple replicas *concurrently* for the same proposal, to increase the chance of the proposal being successful. Concurrent proposals do not destructively interfere: replicas *work together* to decide one of the proposed values.

In short, QuePaxa has three advantages over Raft for our purposes:

1. Because there is no required leader, the system *never* becomes unavailable or degraded due to a single replica (the leader) being down, unavailable, or degraded. Clients can perform writes as long as they can contact some healthy replica (anywhere in the world).
2. Because there is no leader, there are no leader elections that degrade the system. And concurrent proposals made by different replicas *constructively* interfere, unlike Raft's leadership elections. This is ideal for Cloudflare's network, in which latencies can vary wildly.
3. QuePaxa was designed for a less reliable network environment ("asynchrony"), and for networks in which an imaginary adversary can launch targeted attacks on replica connections. The authors found that it maintains much higher (~10x) throughput than Raft and Multi-Paxos during such conditions. These conditions more accurately resemble our own network than the conditions other algorithms assume.

We will save the full description of QuePaxa for another post. Major shoutout to the authors of the QuePaxa paper for being available for feedback and questions about their work.

Assessing Meerkat's performance [🔗](#)

Meerkat has limitations. It is *not* designed to create general-purpose data systems like databases.

All consensus algorithms come with a cost: lots of round-trips. QuePaxa in particular takes one to three round trips (usually, although it can take more) between the initial proposer and a majority of replicas to decide on a proposal and add an event to the log. The difference is with the leader. It takes one if the leader is proposing (+ an extra broadcast to notify replicas of the decision) and three if a non-leader is proposing (+ extra broadcast). If multiple replicas

make proposals at the same time, it can take more. These communication costs point to the important performance limitation of consensus algorithms in general: proposal decision latency is proportional to the latency between some majority of replicas. So if your replicas are far from one another, latency will increase — there's no getting around that.

At first glance, it seems Meerkat's write *and* read latency will be quite poor. Especially if all writes and reads (for consistency) must go through the log, and thus require so many round trips.

But there are a few ways to squeeze better performance out of Meerkat:

1. Because developers have control over where their replicas live, they can choose to move replicas closer together, reducing round-trip latency (only applicable for services that don't need truly global distribution).
2. Writes can be batched. So if a replica receives 10 writes in a span of 10ms, it can place all of those in a single proposal, improving throughput.
3. Not all reads must trigger a consensus round. If a developer is OK with reading *stale* (but never inconsistent) data, they can read from *any* replica's local data.
4. Multiple operations can be bundled into a single consensus round. For instance, our key-value store supports compare-and-swap-style writes in which writes execute only if a value has not changed since it was read. (In fact, it supports general transactions.)

Still, Meerkat's fundamental latency limitations remain, especially when it is run at global scale, as it was designed to do. These limitations make it perfect,

in the short term, for control plane information that is written infrequently but must remain consistent.

What's next [↗](#)

Meerkat is not deployed to production, but we have run multiple proofs-of-concept with up to 50 replicas distributed around the world, to great success. **Leaders in our proof-of-concept clusters constantly fail, and the cluster keeps operating with no increase in error-rate.**

We have a lot more to say about Meerkat. Over the course of the next year we'll be writing Meerkat posts that discuss how QuePaxa really works, how we're formally verifying some of our Rust implementation, how bootstrapping and cluster management works, how we find optimal replica placement, how we use deterministic simulation testing to find bugs, and more. We'll also be preparing a manuscript for peer-review!

Follow along on the Cloudflare Blog as Meerkat progresses, and check out more of our projects at [Cloudflare Research](#).

Discuss on Hacker News

[Research](#) [Network](#) [Database](#) [Distributed Systems](#) [Meerkat](#)

Follow on X

James Larisch | [@jameslarisch](#)

Cloudflare | [@cloudflare](#)

RELATED POSTS

July 09, 2026

Why we cannot wait for better post-quantum signature algorithms

NIST is advancing nine new post-quantum signature algorithms as potential candidates for future standardization. We take a closer look at all of them, and argue that while they are in the works and show great potential, we should use ML-DSA for now — the best one currently available. ...

By Bas Westerbaan, Christopher Patton

[Post-Quantum](#), [Security](#), [Research](#), [Cryptography](#)

June 18, 2026

Build your own vulnerability harness

We break down the technical architecture behind our multi-stage vulnerability discovery harness and automated triage loop. Learn how we manage state controls, squash false positives through adversarial review, and route around LLM context limits....

By Dan Jones, Alexandra Godoi, Grant Bourzikas

[Security](#), [AI](#), [Engineering](#), [Research](#), [Vulnerabilities](#)

May 14, 2026

Our billing pipeline was suddenly slow. The culprit was a hidden bottleneck in ClickHouse

When a partitioning change to our petabyte-scale ClickHouse cluster caused critical billing jobs to stall, standard metrics showed no obvious errors. This post explores how we identified severe lock contention in ClickHouse's query planner and built upstream patches to fix it....

By James Morrison, Christian Endres

[ClickHouse](#), [Engineering](#), [Performance](#), [Database](#), [Open Source](#)

April 21, 2026

Moving past bots vs. humans

As AI assistants and privacy proxies challenge the capabilities of traditional bot detection, the Web needs new models for accountability. We believe that control should remain with the client, and that an open ecosystem of anonymous credentials is key to preserving user privacy while protecting origins from abuse....

By Thibault Meunier

[Research](#), [Bots](#), [Privacy](#), [Standards](#), [Better Internet](#), [AI](#)



© 2026 Cloudflare, Inc. | [Privacy Policy](#) | [Terms of Use](#) | [Report Security Issues](#) | [Your Privacy Choices](#) | [Trademark](#)